Task 14: Sobel Edge Detection

Evaluates spatial intensity gradients using horizontal and vertical 3×3Sobel operators to detect object contours.

Clear Answer:

এই প্রোগ্রামটির নাম Sobel Edge Detection using Python। এটি Digital Image Processing-এর একটি গুরুত্বপূর্ণ Edge Detection Technique।

এই প্রোগ্রামের মূল কাজ হলো একটি Image-এর মধ্যে থাকা Edges, Boundaries এবং Object-এর Outline শনাক্ত করা।

তোমার দেওয়া ছবিতে দেখা যাচ্ছে, প্রথমে Original Cat Image রয়েছে, দ্বিতীয় ছবিতে Horizontal Gradient (Gx) এবং তৃতীয় ছবিতে সম্পূর্ণ Sobel Edge Detection Result দেখানো হয়েছে।

প্রোগ্রামটি মূলত ৪টি কাজ করে:

1. Original Image-কে Grayscale-এ Convert করে।
2. Sobel X এবং Sobel Y Kernel ব্যবহার করে Gradient Calculate করে।
3. Gradient Magnitude ব্যবহার করে Edge Detect করে।
4. Result Normalize করে Original এবং Edge Image Display করে।

1. Sobel Edge Detection কী?

Sobel Edge Detection হলো একটি Edge Detection Algorithm, যা Image-এর Pixel Intensity-এর পরিবর্তন শনাক্ত করে Object-এর Boundary বা Edge বের করে।

সহজভাবে, একটি ছবিতে যেখানে Pixel Value দ্রুত পরিবর্তন হয়, সেখানে সাধারণত Edge থাকে।

উদাহরণ:

- একটি বিড়ালের চোখের Boundary।
- মুখের Outline।
- নাকের চারপাশের Edge।
- ছবির বিভিন্ন Object-এর Border।

Sobel Operator দুটি Kernel ব্যবহার করে:

1. Gx (Horizontal Gradient):

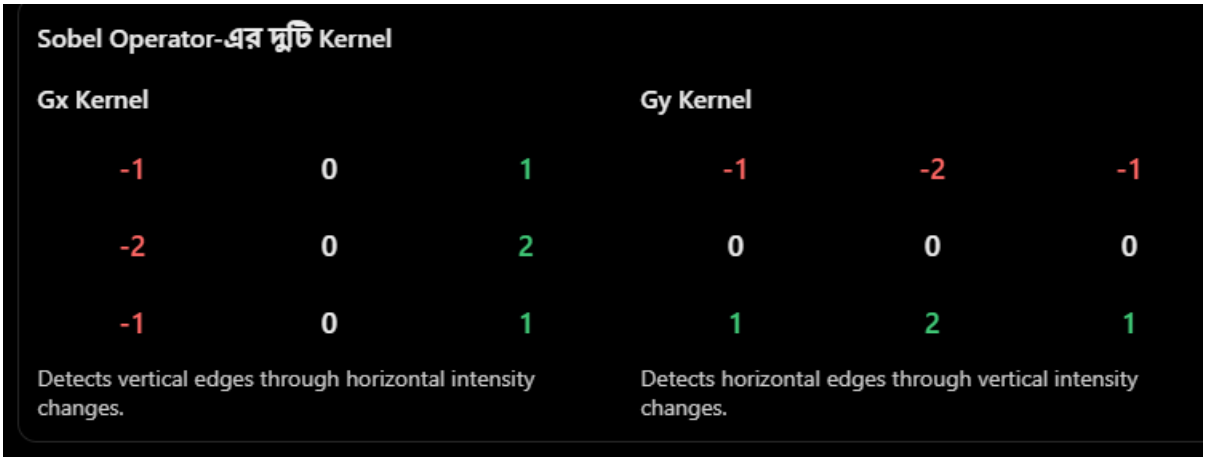
এটি X-direction বা বাম থেকে ডানে Pixel Intensity-এর পরিবর্তন শনাক্ত করে। ফলে সাধারণত Vertical Edges শনাক্ত হয়।

2. Gy (Vertical Gradient):

এটি Y-direction বা উপর থেকে নিচে Pixel Intensity-এর পরিবর্তন শনাক্ত করে। ফলে সাধারণত Horizontal Edges শনাক্ত হয়।

Sobel Operator-এর দুটি Kernel

Gx Kernel



2. Complete Corrected Python Code

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

image = Image.open("PNG/Task_14_Sobel_Edge_Detection.png").convert("L")
img = np.array(image, dtype=np.float64)

rows, cols = img.shape

Gx = np.array([
    [-1, 0, 1],
    [-2, 0, 2],
    [-1, 0, 1]
])

Gy = np.array([
    [-1, -2, -1],
    [ 0,  0,  0],
    [ 1,  2,  1]
])

gradient_x = np.zeros((rows, cols))
```

```
gradient_y = np.zeros((rows, cols))
edge_image = np.zeros((rows, cols))

for i in range(1, rows - 1):
    for j in range(1, cols - 1):

        sum_x = 0
        sum_y = 0

        for m in range(-1, 2):
            for n in range(-1, 2):

                pixel = img[i + m, j + n]

                sum_x += pixel * Gx[m + 1, n + 1]
                sum_y += pixel * Gy[m + 1, n + 1]

        gradient_x[i, j] = sum_x
        gradient_y[i, j] = sum_y
for i in range(rows):
    for j in range(cols):

        edge_image[i, j] = np.sqrt(
            gradient_x[i, j] ** 2 +
            gradient_y[i, j] ** 2
        )
max_value = np.max(edge_image)

if max_value > 0:
    edge_image = (edge_image / max_value) * 255

edge_image = edge_image.astype(np.uint8)
```

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 3, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(gradient_x, cmap="gray")
plt.title("Horizontal Gradient (Gx)")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(edge_image, cmap="gray")
plt.title("Sobel Edge Detection")
plt.axis("off")

plt.tight_layout()
plt.show()
```

3. Code Explanation (Step by Step)

Step 1: Import Libraries

import numpy as npimport matplotlib.pyplot as pltfrom PIL import Image

এখানে তিনটি Library Import করা হয়েছে।

- numpy: Array এবং Mathematical Calculation করার জন্য।
- matplotlib.pyplot: Image Display করার জন্য।
- PIL: Image Open এবং Grayscale Conversion করার জন্য।

Step 2: Load Grayscale Image

image = Image.open("PNG/Task_14_Sobel_Edge_Detection.png").convert("L")img = np.array(image, dtype=np.float64)

প্রথম লাইনে Image.open() ব্যবহার করে নির্দিষ্ট Location থেকে Image Load করা হয়েছে।

এরপর convert("L") ব্যবহার করে Image-কে Grayscale Format-এ Convert করা হয়েছে।

দ্বিতীয় লাইনে np.array() ব্যবহার করে Image-কে NumPy Array-তে Convert করা হয়েছে।

এখানে dtype=np.float64 ব্যবহার করা হয়েছে যাতে Mathematical Calculation-এর সময় Floating Point Value নিয়ে কাজ করা যায়।

মনে রাখবে: Grayscale Image-এর Pixel Value সাধারণত 0 থেকে 255 পর্যন্ত হয়।

Step 3: Get Image Dimensions

rows, cols = img.shape

এখানে Image-এর Dimension বের করা হয়েছে।

- rows = Image-এর Height।
- cols = Image-এর Width।

এই Dimension ব্যবহার করে পরবর্তী Loop চালানো হবে।

Step 4: Define Sobel X Kernel

Gx = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]])

এখানে Sobel X Kernel তৈরি করা হয়েছে।

এটি Image-এর Horizontal Direction বা X-direction-এর Intensity Change শনাক্ত করে।

অর্থাৎ বাম থেকে ডানে Pixel Value-এর পরিবর্তন পরীক্ষা করে।

এই Kernel সাধারণত Vertical Edge শনাক্ত করতে সাহায্য করে।

উদাহরণ: একটি Object-এর বাম এবং ডান পাশের Boundary।

Step 5: Define Sobel Y Kernel

Gy = np.array([[-1, -2, -1], [0, 0, 0], [1, 2, 1]])

এখানে Sobel Y Kernel তৈরি করা হয়েছে।

এটি Image-এর Vertical Direction বা Y-direction-এর Intensity Change শনাক্ত করে।

অর্থাৎ উপর থেকে নিচে Pixel Value-এর পরিবর্তন পরীক্ষা করে।

এই Kernel সাধারণত Horizontal Edge শনাক্ত করতে সাহায্য করে।

উদাহরণ: একটি Object-এর উপরের এবং নিচের Boundary।

Step 6: Create Empty Gradient Images

gradient_x = np.zeros((rows, cols))gradient_y = np.zeros((rows, cols))edge_image = np.zeros((rows, cols))

এখানে তিনটি Blank Matrix তৈরি করা হয়েছে।

- gradient_x: Gx-এর Result রাখবে।
- gradient_y: Gy-এর Result রাখবে।
- edge_image: Final Edge Detection Result রাখবে।

np.zeros() ব্যবহার করার কারণে শুরুতে সব Pixel Value 0 থাকবে।

Step 7: Apply Sobel Operator

for i in range(1, rows - 1): for j in range(1, cols - 1):

এখানে Nested Loop ব্যবহার করা হয়েছে।

প্রথম Loop প্রতিটি Row Access করে।

দ্বিতীয় Loop প্রতিটি Column Access করে।

তবে এখানে Loop শুরু হয়েছে 1 থেকে এবং শেষ হয়েছে rows - 1 বা cols - 1-এর আগে।

কারণ 3×3 Kernel ব্যবহার করার জন্য Center Pixel-এর চারপাশে এক Pixel করে Neighbor প্রয়োজন।

তাই Border Pixel-গুলো এই Loop-এ Process করা হচ্ছে না।

Step 8: Initialize Sum Variables

sum_x = 0 sum_y = 0

এখানে দুটি Variable তৈরি করা হয়েছে।

- sum_x: Gx Kernel-এর Calculation Store করবে।
- sum_y: Gy Kernel-এর Calculation Store করবে।

প্রতিটি নতুন Pixel Process করার আগে দুটি Variable-এর Value 0 করা হচ্ছে।

Step 9: Access 3×3 Neighborhood

for m in range(-1, 2): for n in range(-1, 2):

এখানে আরও দুটি Nested Loop ব্যবহার করা হয়েছে।

এই Loop-এর মাধ্যমে Current Pixel-এর আশেপাশের 3×3 Neighborhood Access করা হচ্ছে।

m এবং n-এর Value হবে:

-1, 0, 1

অর্থাৎ মোট 9টি Pixel Access করা হবে।

Step 10: Get Pixel Value

pixel = img[i + m, j + n]

এখানে Original Image থেকে Current Neighborhood-এর Pixel Value নেওয়া হয়েছে।

- i + m = Neighboring Row Position।
- j + n = Neighboring Column Position।

এভাবে একে একে 9টি Pixel Value পাওয়া যাবে।

Step 11: Calculate Horizontal and Vertical Gradients

sum_x += pixel * Gx[m + 1, n + 1]sum_y += pixel * Gy[m + 1, n + 1]

Most Important Code Section

এটি Sobel Edge Detection-এর মূল Calculation।

প্রথম লাইনে Pixel Value-এর সঙ্গে Gx Kernel-এর Corresponding Value Multiply করা হচ্ছে।

এরপর সব Result যোগ করে sum_x-এ রাখা হচ্ছে।

দ্বিতীয় লাইনে একইভাবে Gy Kernel ব্যবহার করে Vertical Gradient Calculate করা হচ্ছে।

অর্থাৎ:

- sum_x = Horizontal Intensity Gradient।
- sum_y = Vertical Intensity Gradient।

সহজভাবে, Image-এর দুই Direction-এর Intensity Change বের করা হচ্ছে।

Step 12: Store Gradient Values

gradient_x[i, j] = sum_xgradient_y[i, j] = sum_y

এখানে Calculate করা Gradient Value-গুলো আলাদা Matrix-এ Store করা হচ্ছে।

gradient_x-এ Gx-এর Result এবং gradient_y-এ Gy-এর Result রাখা হচ্ছে।

এর ফলে পরবর্তী সময়ে দুটি Gradient একসঙ্গে ব্যবহার করে Final Edge Image তৈরি করা যাবে।

Step 13: Calculate Gradient Magnitude

for i in range(rows): for j in range(cols): edge_image[i, j] = np.sqrt(gradient_x[i, j] ** 2 + gradient_y[i, j] ** 2)

এটি আমাদের প্রোগ্রামের আরেকটি গুরুত্বপূর্ণ অংশ।

এখানে Gx এবং Gy-এর সাহায্যে প্রতিটি Pixel-এর Edge Strength বা Gradient Magnitude বের করা হয়েছে।

Mathematical Formula:

$$G = \sqrt{G_x^2 + G_y^2}$$

এখানে:

- Gx = Horizontal Gradient।
- Gy = Vertical Gradient।
- G = Final Gradient Magnitude।

যেখানে Gradient Magnitude বেশি, সেখানে সাধারণত Strong Edge থাকে।

যেখানে Gradient Magnitude কম, সেখানে Edge দুর্বল অথবা অনুপস্থিত।

Step 14: Normalize Edge Image

max_value = np.max(edge_image)if max_value > 0: edge_image = (edge_image / max_value) * 255

এখানে প্রথমে Edge Image-এর Maximum Pixel Value বের করা হয়েছে।

এরপর Maximum Value দিয়ে প্রতিটি Pixel Value ভাগ করে 255 দিয়ে Multiply করা হয়েছে।

একে Normalization বলা হয়।

এর ফলে Pixel Value-গুলো 0 থেকে 255 Range-এ নিয়ে আসা হয়।

এতে Final Edge Image Display করা সহজ হয়।

Step 15: Convert into 8-bit Image

edge_image = edge_image.astype(np.uint8)

এখানে astype(np.uint8) ব্যবহার করে Edge Image-কে 8-bit Unsigned Integer Format-এ Convert করা হয়েছে।

এখন Pixel Value 0 থেকে 255-এর মধ্যে থাকবে।

Step 16: Display Original and Edge Images

plt.figure(figsize=(12, 5))

এখানে Display Window-এর Size নির্ধারণ করা হয়েছে।

এরপর:

plt.subplot(1, 3, 1)

ব্যবহার করে 1 Row এবং 3 Columns-এর Layout তৈরি করা হয়েছে।

তিনটি Image Display করা হয়েছে:

1. Original Image।
2. Horizontal Gradient (Gx)।
3. Sobel Edge Detection।

এখানে:

plt.imshow(img, cmap="gray")plt.title("Original Image")plt.axis("off")

Original Image Display করা হয়েছে।

একইভাবে দ্বিতীয় Subplot-এ Gx Gradient এবং তৃতীয় Subplot-এ Final Edge Image Display করা হয়েছে।

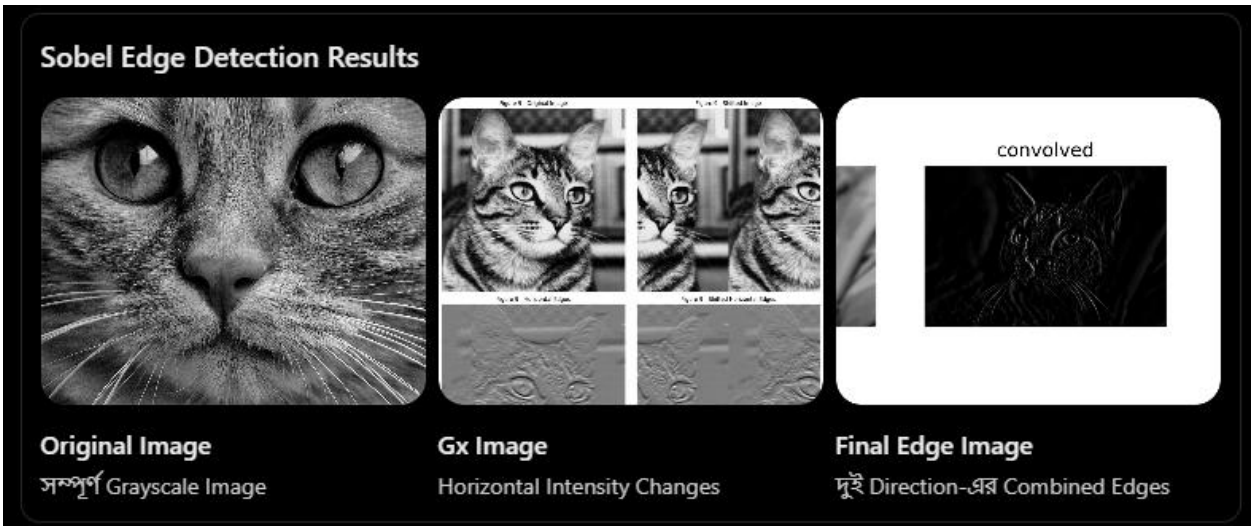
সবশেষে:

plt.tight_layout()plt.show()

tight_layout() Image-গুলোর মধ্যে Spacing ঠিক করে এবং show() সব Image Display করে।

4. তোমার Output Image-এ কী হয়েছে?

Sobel Edge Detection Results



তোমার ছবিতে:

- Original Image-এ বিড়ালের সম্পূর্ণ মুখ দেখা যাচ্ছে।
- Gx Image-এ বাম থেকে ডানে Intensity Change-এর Result দেখা যাচ্ছে।
- Final Sobel Image-এ চোখ, নাক, মুখ এবং লোমের বিভিন্ন Boundary শনাক্ত হয়েছে।

একটি গুরুত্বপূর্ণ বিষয়: শুধু Gx ব্যবহার করলে সব Direction-এর Edge পাওয়া যাবে না। তাই Gx এবং Gy একসঙ্গে ব্যবহার করে Final Gradient Magnitude Calculate করা হয়েছে।

5. Exam-এ শিক্ষককে যেভাবে বুঝাবে

নিচের Paragraph-টি ভালোভাবে বুঝে রাখবে।

Exam Preparation

বাংলায় ব্যাখ্যা:

প্রথমে আমি প্রয়োজনীয় Library Import করেছি এবং Original Image-টি Grayscale Format-এ Load করেছি। এরপর Image-এর Height এবং Width বের করেছি।

তারপর দুটি Sobel Kernel তৈরি করেছি, যেগুলো হলো Gx এবং Gy। Gx ব্যবহার করে Horizontal Direction-এর Intensity Change এবং Gy ব্যবহার করে Vertical Direction-এর Intensity Change বের করেছি।

পরবর্তীতে Nested Loop ব্যবহার করে Image-এর প্রতিটি Pixel-এর 3×3 Neighborhood Access করেছি। প্রতিটি Pixel Value-এর সঙ্গে Gx এবং Gy Kernel-এর Corresponding Value Multiply করে Gradient Calculate করেছি।

এরপর দুটি Gradient-এর Square যোগ করে Square Root নেওয়ার মাধ্যমে Gradient Magnitude বের করেছি। এই Magnitude ব্যবহার করে Image-এর বিভিন্ন Edge শনাক্ত করেছি।

সবশেষে Edge Image Normalize করে 0 থেকে 255 Range-এ নিয়ে এসেছি এবং Original Image, Gx Gradient ও Final Sobel Edge Image পাশাপাশি Display করেছি।

Copy Exam Explanation

6. Important Viva Questions and Answers

Q1. What is Sobel Edge Detection?

Answer: Sobel Edge Detection is an image processing technique used to detect edges by calculating changes in pixel intensity.

বাংলা: Pixel Intensity-এর পরিবর্তন Calculate করে Image-এর Edge শনাক্ত করার পদ্ধতি।

Q2. What is Gx?

Answer: Gx calculates the horizontal intensity gradient and helps detect vertical edges.

Q3. What is Gy?

Answer: Gy calculates the vertical intensity gradient and helps detect horizontal edges.

Q4. Why do we use two Kernels?

Answer: We use two Kernels to detect intensity changes in both horizontal and vertical directions.

Q5. What is Gradient Magnitude?

Answer: Gradient Magnitude represents the overall strength of an edge.

Q6. What is the formula of Gradient Magnitude?

Answer:

$$G = \sqrt{G_x^2 + G_y^2}$$

Q7. What does np.sqrt() do?

Answer: It calculates the square root of a number or array.

Q8. What is the purpose of np.max()?

Answer: It finds the maximum value in an array.

Q9. Why do we use Normalization?

Answer: To scale the edge values into the 0–255 range for image display.

Q10. Why do we use Grayscale Image?

Answer: Edge detection becomes simpler because we work with a single intensity channel instead of three color channels.

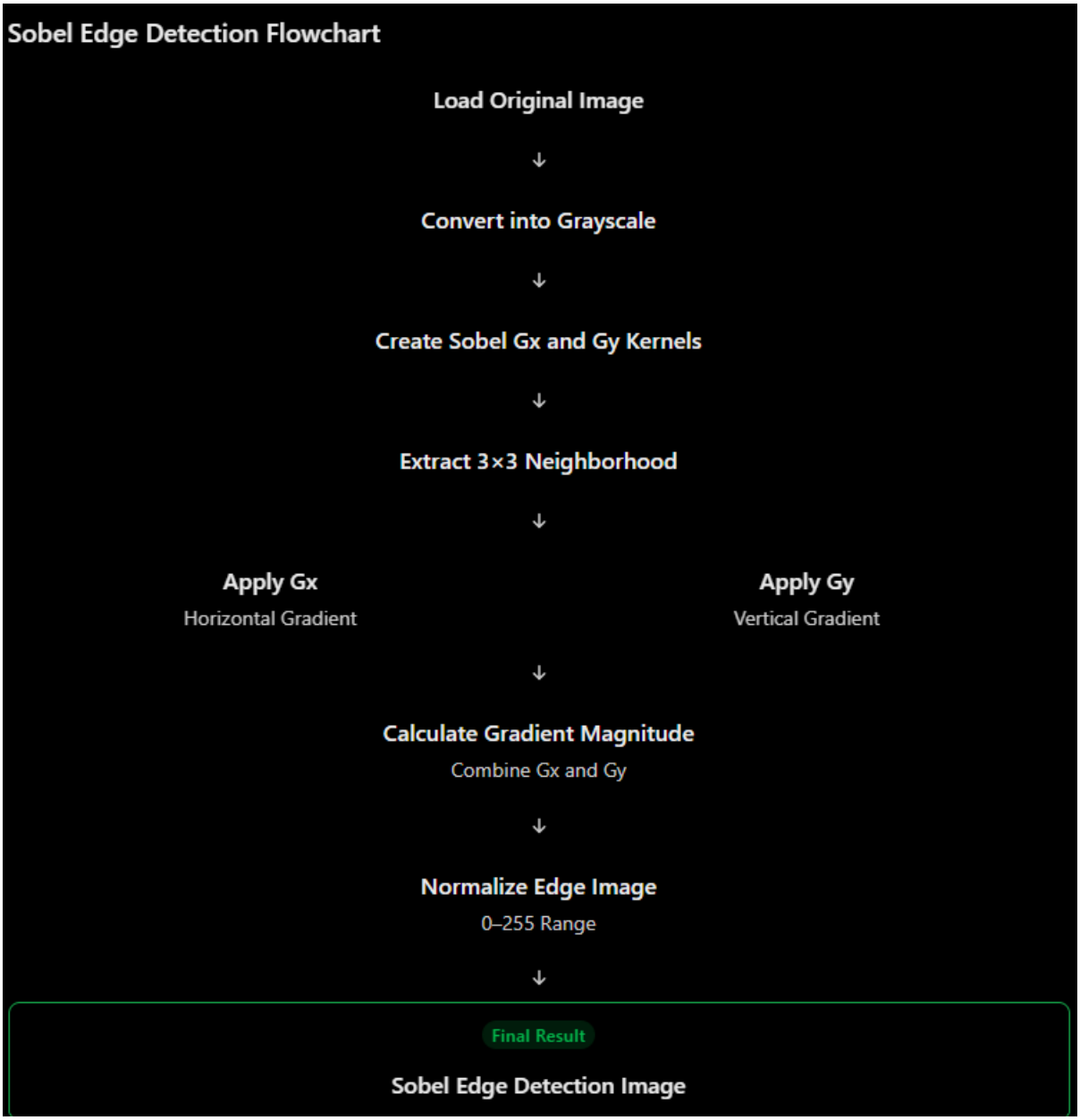
Q11. What is the difference between Gx and Gy?

Answer: Gx detects changes along the horizontal direction, while Gy detects changes along the vertical direction.

Q12. What happens if Gradient Magnitude is high?

Answer: It generally indicates a strong edge or a significant change in pixel intensity.

7. Complete Working Process



Key Caveats:

1. Gx এবং Gy আলাদাভাবে সম্পূর্ণ Edge Detection Result দেয় না। দুটি Combine করে Final Result তৈরি করা হয়।
2. Gx Horizontal Intensity Change শনাক্ত করে, কিন্তু এর মাধ্যমে সাধারণত Vertical Edges দেখা যায়।
3. Gy Vertical Intensity Change শনাক্ত করে, কিন্তু এর মাধ্যমে সাধারণত Horizontal Edges দেখা যায়।
4. তোমার Code-এ Image-এর Outer Boundary Process করা হয়নি। তাই Border Pixel-গুলো Zero রয়েছে।

5. Sobel Operator Noise-এর প্রতিও Sensitive হতে পারে। প্রয়োজনে আগে Gaussian Blur ব্যবহার করা যায়।

পরীক্ষার জন্য সবচেয়ে গুরুত্বপূর্ণ ৫টি Code Section:

```
Gx = np.array([
    [-1, 0, 1],
    [-2, 0, 2],
    [-1, 0, 1]
])

Gy = np.array([
    [-1, -2, -1],
    [0, 0, 0],
    [1, 2, 1]
])

sum_x += pixel * Gx[m + 1, n + 1]
sum_y += pixel * Gy[m + 1, n + 1]

edge_image[i, j] = np.sqrt(
    gradient_x[i, j] ** 2 +
    gradient_y[i, j] ** 2
)

edge_image = (edge_image / max_value) * 255
```

এই অংশগুলো বুঝতে পারলে Sobel Edge Detection-এর মূল Algorithm ব্যাখ্যা করতে পারবে।

মনে রাখার সহজ Sequence:

Grayscale → Gx → Gy → Gradient Magnitude → Normalization → Edge Image